

Disciplina:

CI/CD, PIPELINES E TESTES AUTOMATIZADOS

Professor: Wander Maia da Silva



Plano de ensino



Objetivo da Disciplina

Desenvolver no aluno a capacidade de compreender, projetar e implementar pipelines de CI/CD completos, integrando testes automatizados, segurança e observabilidade em ambientes modernos.

Ao final, o aluno estruturará fluxos de entrega contínua com qualidade, segurança e rastreabilidade, utilizando estratégias de deployment e práticas de GitOps.

Metodologia

- Aulas expositivo-dialogadas e elaboração de apostilas.
- Demonstrações práticas e discussões sobre casos reais.

Sobre o professor:



Wander Maia

Formação Acadêmica

- Graduação em **Sistemas de Informação** (PUC Minas)
- **MBA em DevOps & SRE e Arquitetura de Software e Soluções**
- Pós-Graduação em **Desenvolvimento Avançado em Go**

Interesses

- Entusiasta de **Go / Golang**
- Fã de **Rock & Heavy Metal**

Experiência Profissional

- **13 anos de experiência** em TI
- **Analista DevOps Sênior** na ForceOne IT (parceira AWS)
- Especialista em **AWS, Linux/Unix e Kubernetes**

LinkedIn

<https://www.linkedin.com/in/wandermaia/>

Certificações AWS



Avaliação

A disciplina totaliza **100 pontos**, distribuídos em 3 atividades no **Canvas**:

Unidade 1 → 30 pontos

Unidade 2 → 30 pontos

Unidade 3 → 40 pontos

Cada atividade será liberada no Canvas após o bloco correspondente. Sem prova tradicional – a avaliação é prática e contínua.

Objetivos da Unidade 1



Conceitos de CI/CD

Explicar CI/CD e diferenciar Entrega Contínua de Deploy Contínuo



Mercado e Ferramentas

Comparar as principais plataformas do mercado



Estrutura de Pipeline

Descrever as etapas de um pipeline



Estratégias de Deployment

Distinguir estratégias de deployment e seus trade-offs



GitOps

Entender GitOps e sua integração com CI/CD



Boas Práticas

Aplicar boas práticas no design de pipelines

Agenda



01. O que é CI/CD?

Fundamentos e conceitos essenciais da automação.



02. Plataformas de CI/CD

Visão geral das ferramentas líderes do mercado.



03. Anatomia de um Pipeline

Desconstruindo as fases e estágios de execução.



04. Estratégias de Deployment

Blue/Green, Canary e outras formas de entrega.



05. GitOps

A evolução da infraestrutura como código.



06. Boas Práticas

Design e manutenção para alta performance.

1. O que é CI/CD?

O Problema: Integration Hell



Isolamento Prolongado

Times trabalham semanas em branches separados



O "Big Bang" de Sexta-feira

Tudo vai para a main ao mesmo tempo



Integração Lenta

Integrar, testar e subir: um dia inteiro perdido



Rastreabilidade Zero

Quando quebra, ninguém sabe o que causou

CI/CD é a resposta da engenharia de software para esse problema

Integração Contínua (CI)



Integrar código ao repositório principal com alta frequência

Cada integração dispara build e testes automatizados.

Princípio central: quanto menor o lote, menor o risco.

Princípios fundamentais:



Repositório único | Build e Testes automatizados



Falha visível para todo o time



Build rápido (minutos, não horas)



Correção é prioridade máxima

Entrega Contínua vs. Deploy Contínuo

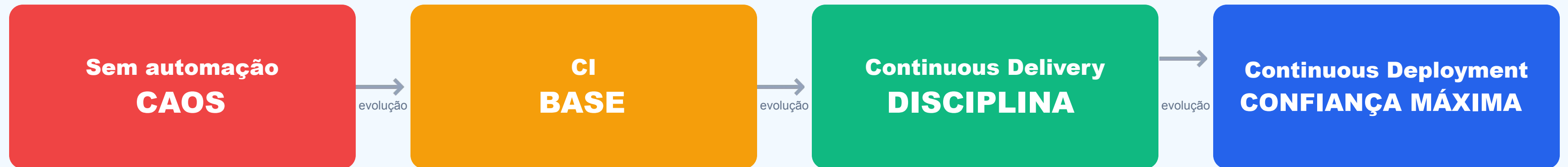
Continuous Delivery

- ✓ Software sempre em estado “**deployável**”
- 👉 Deploy para produção ainda é acionado **manualmente**
- 🏢 Decisão de **negócio**, não limitação técnica

Continuous Deployment

- 🔄 Qualquer mudança que passa no pipeline vai **automaticamente** para produção
- 🚀 Netflix, Amazon, Facebook: centenas de deploys por dia
- ⓘ Atenção: o mercado usa “**CD**” para os dois. Entenda o contexto

Espectro de Maturidade



Principais Considerações:



- Não existe nível "certo" para todos
- Depende: tipo de produto, maturidade do time e requisitos de negócio

Benefícios do CI/CD



Velocidade de entrega

Releases pequenas e frequentes para um ciclo de feedback contínuo.



Detecção precoce de bugs

Encontrar erros cedo é significativamente mais barato do que em produção.



Redução de risco

Menos mudanças por release facilitam rollback e garantem estabilidade.



Feedback rápido

Desenvolvedores sabem em minutos se o novo código quebrou algo.



Confiança e previsibilidade

O time tem segurança de que o processo de automação funciona.



Colaboração Dev + Ops

O pipeline torna-se um artefato compartilhado e transparente.

Desafios do CI/CD



Resistência cultural

Como endereçar:

Começar pequeno, mostrar resultados



Cobertura de testes

Como endereçar:

Testes fazem parte do pipeline, não são extra



Pipelines lentos

Como endereçar:

Paralelismo, caching, seleção inteligente



Ambientes diferentes

Como endereçar:

IaC, containers, paridade de ambientes



Segurança negligenciada

Como endereçar:

DevSecOps desde o início



Dívida técnica

Como endereçar:

Migração incremental, não big bang

CI/CD no Contexto DevOps

A Espinha Dorsal do DevOps

CI/CD é a espinha dorsal técnica que operacionaliza a cultura DevOps em fluxos automatizados.

Fluxo DevOps Contínuo

Plan → **Code** → **Build** → **Test** → **Release** → **Deploy** → **Operate** → **Monitor**

CI (Integração Contínua)

Foco na automação da build e validação de código.

- Build
- Test
- Release (Artefato)

CD (Entrega/Implantação)

Foco na automação da entrega de valor ao usuário.

- Deploy
- Operate
- Monitor

Conceitos Relacionados: *GitOps* • *DevSecOps* • *Observabilidade* • *DORA Metrics*

2. Plataformas de CI/CD

As 4 Principais Plataformas



GitHub Actions

MODELO

SaaS / self-hosted

IDEAL PARA

Open source, startups



GitLab CI/CD

MODELO

SaaS / self-hosted

IDEAL PARA

Times DevOps completos



Jenkins

MODELO

Self-hosted

IDEAL PARA

Empresas com infra legada



Azure DevOps

MODELO

SaaS / self-hosted

IDEAL PARA

Times Azure / .NET

*Não existe "melhor" em absoluto, existe o mais **adequado** para o contexto*

GitHub Actions



Configuração

Arquivos YAML em `.github/workflows/`



Gatilhos

Disparado por eventos do GitHub (push, PR, schedule...)

Conceitos-chave

Workflow: Arquivo YAML completo

Job: Unidade de execução em runner isolado

Step: Passo dentro de um job

Action: Componente reutilizável (+20k no marketplace)

Runner: Máquina virtual onde o job executa

Pontos fortes: *integração nativa com GitHub, curva de aprendizado baixa*

GitLab CI/CD



Configuração

Arquivo `.gitlab-ci.yml` na raiz do repositório



Plataforma Integrada

Repositório + CI/CD + Registry + Monitoramento

Conceitos-chave

Stages: Grupos de jobs que rodam em sequência

Artifacts: Arquivos passados entre jobs da pipeline

Environments: Ambientes rastreados pela plataforma

GitLab Runner: Agente responsável pela execução dos jobs

Pontos fortes: *Solução all-in-one, excelente on-premise e Auto DevOps integrado*

Jenkins



Configuração

Jenkinsfile (**Groovy**) ou Interface Web



Extensibilidade

Open source desde 2004, 1.800+ plugins

Conceitos-chave

Controller: Servidor central que orquestra

Agent/Node: Máquina onde os jobs executam

Plugin: Extensão do sistema

Pontos fortes: Máxima flexibilidade, custo zero

Limitações: Gestão da infraestrutura, aprendizado íngreme

Analogia: *Jenkins é ter sua própria oficina. Actions é usar uma já equipada.*

Azure DevOps



Configuração

Arquivo `azure-pipelines.yml` (YAML)



Plataforma Microsoft

Repos + Pipelines + Boards + Artifacts

Conceitos-chave

Stage: Agrupamento de jobs com dependências

Task: Passo pré-construído disponível no marketplace

Environment: Ambiente com histórico e aprovações nativas

Pontos fortes: *Integração profunda com Azure, aprovações manuais nativas, ideal para .NET*

Quando Usar Cada Plataforma



Código no GitHub

Use o **GitHub Actions** para automação nativa e integrada ao repositório.



Código no GitLab

Aproveite o **GitLab CI/CD** para pipelines robustos integrados ao ciclo de vida.



Código no Azure Repos

O **Azure DevOps** é o parceiro natural para ecossistemas Microsoft e Azure.



On-Premise / Híbrido

Jenkins ou **GitLab self-hosted** oferecem controle total e flexibilidade.

Não há uma solução universal. A escolha ideal depende sempre da sua necessidade específica.

3. Anatomia de um Pipeline

Os 4 Estágios de um Pipeline



Build

Transforma o código-fonte em um artefato executável pronto para o ambiente.



Test

Valida a qualidade, segurança e o comportamento esperado da aplicação.



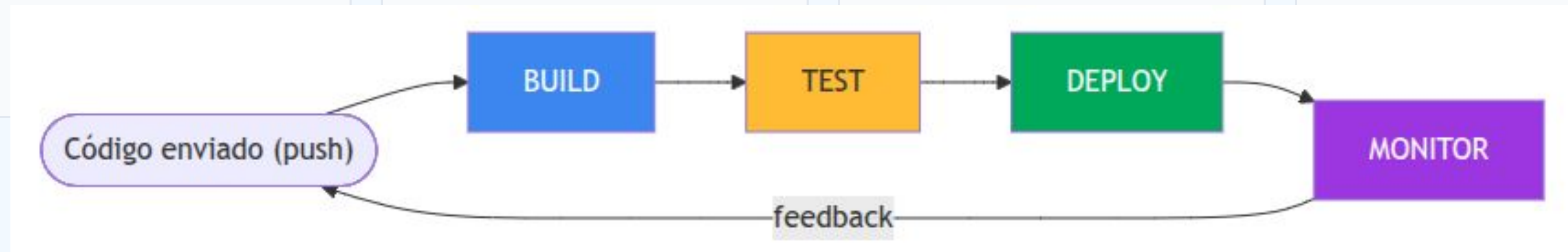
Deploy

Envia o artefato validado para os ambientes de staging ou produção.



Monitor

Observa a saúde do sistema em produção e fecha o loop de feedback.



Estágio de Build

Objetivo: transformar código-fonte em artefato executável

Responsabilidades: Instalar deps, compilar, gerar artefato, linting e análise estática

Java / Kotlin

Compilação + geração de artefatos JAR ou WAR.

Node.js

Instalação de dependências e processo de bundling.

Docker

Execução do docker build para criar imagem de container.

Go

Compilação do código para um binário estático.

Princípio: *se o build falha, o pipeline para. Não testamos código que não compila.*

Estágio de Test - A Pirâmide



E2E (End-to-End)

Poucos, lentos, alto valor. Simulam a jornada completa do usuário real.

Integração

Validam a comunicação entre componentes: Banco de dados, APIs e Filas.

Unitários

Muitos, rápidos e isolados. Foco total na validação da lógica de negócio.

Boas práticas: *rode os mais rápidos primeiro; paralelize suítes independentes*

Falha de teste = falha de build

Outros Tipos de Testes no Pipeline

Smoke tests

"O sistema está de pé?"
verificação rápida pós-deploy
para garantir que as
funcionalidades críticas estão
operacionais.

Performance tests

Analisa o tempo de resposta e o
comportamento do sistema sob
diferentes níveis de carga e
estresse.

Contract tests

Garante que a API continua
compatível com seus
consumidores, validando
contratos de integração.

Mutation tests

Avalia a eficácia da suíte de
testes inserindo falhas no código
para ver se os testes as
detectam.

Estes testes complementam a pirâmide tradicional para garantir resiliência e qualidade em diferentes camadas.

Estágio de Deploy

O que o Deploy faz:



Coleta de Artefato

Busca o artefato gerado e validado no estágio de Build.



Configuração

Aplica env vars e secrets específicas para o ambiente de destino.



Estratégia

Executa o deployment seguindo a estratégia definida (Blue/Green, Canary, etc).



Health Check

Monitora e confirma se o serviço está saudável após a atualização.



Rastreabilidade

Registra o evento de deploy para histórico e auditoria.



Quality Gates



Cobertura de Testes

A cobertura de testes automatizados está acima do limite de X% definido?



Segurança

Existem vulnerabilidades críticas detectadas no scan de segurança?



Performance (SLA)

O tempo de resposta do sistema está dentro dos limites de SLA?



Aprovação Manual

O deploy exige a validação e aprovação de algum membro do time?



Promoção de Ambiente

Verificações automáticas antes de avançar entre ambientes.

Se qualquer gate falhar → pipeline para → artefato não avança

Estágio de Monitor

O pipeline não termina no deploy.



Métricas

Latência, taxa de erros, CPU e memória do sistema.



Logs

Eventos, erros e rastreamento detalhado de requisições.



Traces

Caminho de uma requisição através de múltiplos serviços.



Health checks

Verificação constante se o endpoint /health está respondendo.



Alertas

Notificações automáticas quando thresholds são ultrapassados.

Rollback automático: *se a taxa de erros subir além do threshold → reverte*

Lab 01



Capítulos Relacionados

2 (Plataformas de CI/CD) e 3 (Anatomia de um Pipeline)



Duração Estimada

40 minutos



Objetivo Principal

Criar um repositório no Azure DevOps com aplicação Python e pipeline YAML automatizado.

Atividades do Laboratório

- Provisionar novo repositório no [Azure DevOps](#)
- Adicionar aplicação [Python](#) com testes unitários baseados em *pytest*
- Escrever e configurar o arquivo [azure-pipelines.yml](#)
- Validar execução automática de [Build e Testes](#) a cada Push

Pratique os conceitos de CI/CD em um cenário real de automação de desenvolvimento.

4. Estratégias de Deployment

Visão Geral - 4 Estratégias



Rolling Update

Substituição gradual das instâncias, v1 e v2 coexistem durante a transição.



Blue-Green

Dois ambientes paralelos com virada instantânea de 100% do tráfego.



Canary

Rollout progressivo por percentual de tráfego para validar a nova versão.



Feature Flags

Controle granular de comportamento e visibilidade de features por código.

A escolha da estratégia ideal depende da complexidade da infraestrutura e dos requisitos de disponibilidade do negócio.

Rolling Update

Como funciona

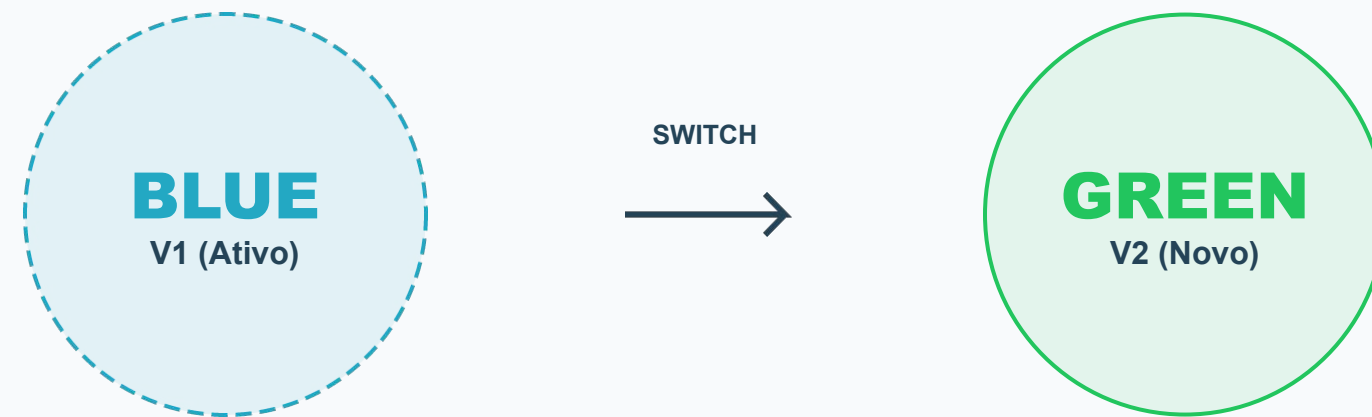
- Substitui instâncias da versão antiga pela nova aos poucos, em lotes.
- Durante a transição: v1 e v2 coexistem recebendo tráfego.
- Padrão nativo do Kubernetes.

Quando usar

- Aplicações stateless (sem estado).
- Mudanças compatíveis com a versão anterior.
- Quando você busca a estratégia mais simples.

Atenção: Se a v2 tiver um bug grave, algumas requisições já terão sido afetadas antes do rollback.

Blue-Green Deployment



- **Dois ambientes idênticos em paralelo:** Blue e Green.
- **Deploy isolado:** Ocorre no Green enquanto Blue atende 100% dos usuários.
- **Virada:** Load Balancer redireciona o tráfego após validação.

Vantagens

- ✓ **Rollback instantâneo:** Redirecionamento imediato para o ambiente anterior.
- ✓ **Zero downtime:** Transição transparente para o usuário final na virada.

Desvantagens

- ! **Custo elevado:** Infraestrutura duplicada para manter ambos ativos.
- ! **Complexidade de dados:** BD deve suportar v1 e v2 simultaneamente.

Ideal para aplicações onde o tempo de inatividade é inaceitável e o custo de infraestrutura é secundário.

Canary Deployment



Fase Inicial
1–5% do tráfego



Progressão
Aumento gradual



Estabilidade
Até 100%



Rollback
Afeta apenas fatia

Progressão Recomendada

- 1–5% (15-30 min)
- 25% (30-60 min)
- 50% (1-2h)
- 100% (Finalizado)

Quando Usar

- **Serviços Críticos:** Onde falhas impactam o negócio.
- **Impacto Incerto:** Funcionalidades novas e complexas.
- **Validação Real:** Necessidade de teste com usuários reais.

Curiosidade: O nome vem dos canários nas minas de carvão, que detectavam gases tóxicos antes dos mineiros.

Feature Flags

O que é: Código vai para produção, mas fica desligado por padrão.

Ativação Dinâmica: Uma flag decide se o recurso está ativo por usuário, grupo, percentual ou região.

Casos de Uso

- A/B Testing
- Rollout Progressivo & Kill Switch
- Beta Interno
- Lançamento por Região

Diferença para Canary

Canary: Separa versões na infraestrutura (dois processos isolados).

Feature Flag: Controla comportamento no código (um binário, caminhos condicionais).

Cuidado: *Flags que nunca são removidas viram dívida técnica — defina data de expiração*

Comparativo das Estratégias

Rolling Update

Rollback
Lento

Custo
Baixo

Complexidade
Baixa

Blue-Green

Rollback
Instantâneo

Custo
Alto (2x)

Complexidade
Média

Canary

Rollback
Rápido

Custo
Médio

Complexidade
Alta

Feature Flags

Rollback
Instantâneo

Custo
Baixo

Complexidade
Média

Na prática as estratégias se combinam — não são mutuamente exclusivas

Lab 02

Capítulos: 2 (CI/CD) e 4 (Deployment)

Duração: 50 minutos

Pré-requisito: Lab 1 concluído (repositório `pipeline-labs` e pipeline ativos).

Objetivo

Instalar agente auto-hospedado e evoluir o pipeline para simular **Rolling Update** e **Blue-Green** via Environments e aprovação manual no Azure DevOps.

Atividades Práticas:

1. Configuração de Self-hosted Agent
2. Implementação de Estratégia Rolling Update
3. Configuração de Blue-Green com Aprovação Manual

Foco: **Azure DevOps Environments** e **Checks de Aprovação**

5. GitOps

Git: Conceitos Essenciais

Antes de GitOps, alinhar o vocabulário do Git:

Repositório

Pasta do projeto com todo o histórico versionado

Commit

"Ponto de salvamento": snapshot das mudanças

Branch

Linha paralela de desenvolvimento

Merge

Junção de mudanças de uma branch com outra

Pull Request (PR)

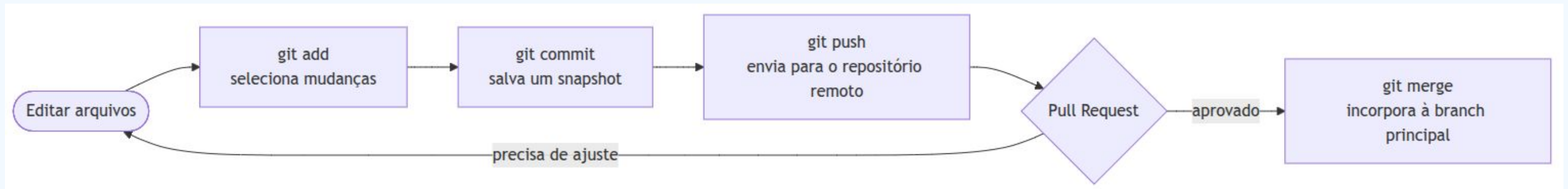
Proposta de mudança enviada para revisão

Revert / Rollback

Desfazer uma mudança, voltando ao histórico

GitOps usa exatamente esses **conceitos**, só que **aplicados à infraestrutura**

Git e Machine Learning



Limitações do Git Tradicional

Git foi desenhado para **arquivos de texto** (código, YAML). Não lida bem com **arquivos grandes e binários** como datasets e checkpoints de modelos.

Soluções Complementares em MLOps

Git LFS (Large File Storage): Armazenamento de grandes arquivos integrado ao Git.

DVC (Data Version Control): Controle de versão especializado para dados e pipelines.

GitOps usa exatamente esses **conceitos**, só que **aplicados à infraestrutura**

O que é GitOps

 **Git como única fonte de verdade** para o estado desejado do sistema.



Declarativo

Estado descrito em YAML/HCL/JSON. Você define o 'quê', não o 'como'.



Versionado

Tudo no Git: histórico completo e capacidade de rollback imediata.



Automático

Pulled automaticamente: um agente no cluster detecta e aplica as mudanças.



Reconciliado

Agente corrige desvios (drift) automaticamente para manter o estado desejado.

Aplicar os princípios de versionamento de código para gerenciar infraestrutura e deploys.

Push-based vs Pull-based

Push-based (CI/CD tradicional)



Risco de Credenciais

O pipeline de CI/CD externo armazena credenciais administrativas para acessar o cluster diretamente.



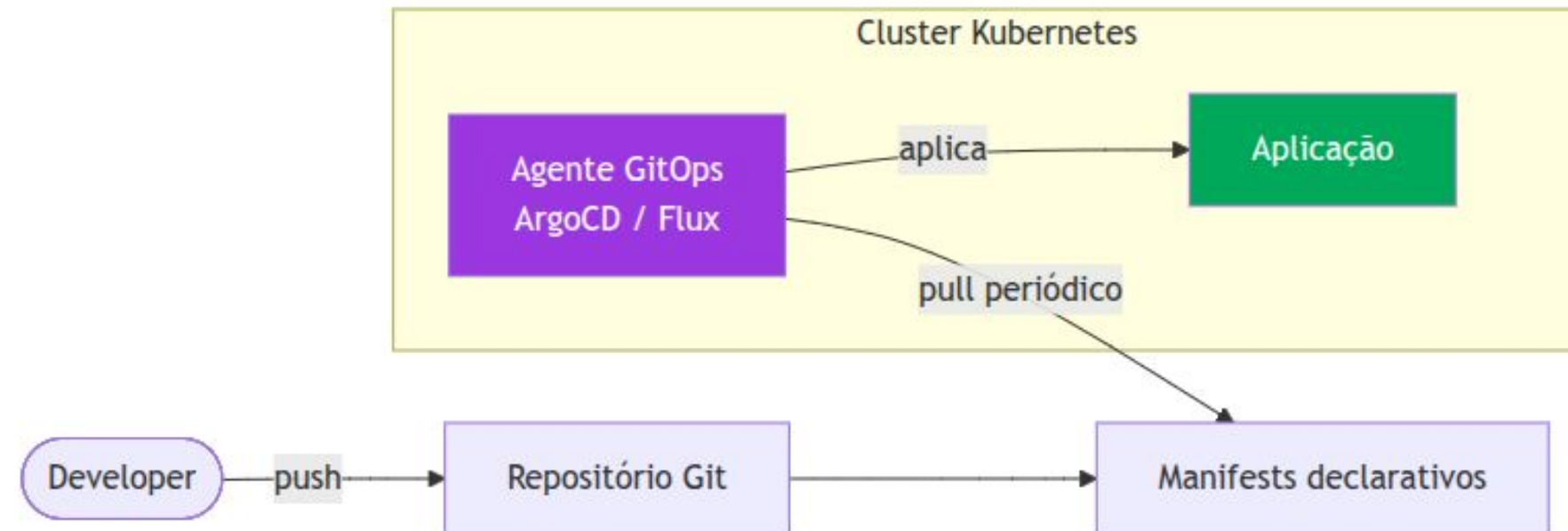
Superfície de Ataque

Se o servidor de CI for comprometido, o atacante obtém acesso total a toda a infraestrutura.

Modelo Push: *Centraliza a vulnerabilidade fora da rede protegida do cluster.*

Push-based vs Pull-based

Pull-based (GitOps)



Segurança Elevada

O pipeline externo não precisa de credenciais ou acesso direto ao cluster, reduzindo a superfície de ataque.

Sincronização Automática

Um agente interno (ArgoCD/Flux) monitora o Git e aplica mudanças e correções de desvio (drift) continuamente.

Vantagem Principal: Pipeline externo isolado sem necessidade de acesso privilegiado ao cluster.

ArgoCD vs Flux

Critério	ArgoCD	Flux
Interface visual	Sim, rica	Não (dashboards externos)
Multi-cluster	Nativo e simples	Suportado, mais configuração
Filosofia	App-centric	GitOps puro, CRD-first
Curva aprendizado	Menor	Maior

ArgoCD

Focado em visibilidade: interface visual rica, árvore de recursos e status em tempo real. Ideal para equipes que priorizam monitoramento visual.

Flux

Focado em automação: 100% declarativo via CRDs, sem interface nativa. Recomendado para ambientes altamente automatizados e "GitOps-first".

CI/CD + GitOps — Fluxo Integrado

01

Push p/ App Repo

Developer realiza o commit e push do código fonte para o repositório da aplicação.

02

Pipeline CI

Execução de Build + Test e push da imagem para o Container Registry.

03

Update Manifests

Pipeline atualiza automaticamente a tag da imagem no repositório de infra/manifestos.

04

Pull do Agente

Agente GitOps (ArgoCD/Flux) detecta mudanças e faz pull do repositório de manifests.

05

Sincronização

O agente aplica as mudanças no Cluster de Produção garantindo o estado desejado.

Auditoria completa: *Quem mudou o quê em produção está no histórico do Git.*

Lab 03

Capítulo Relacionado
5 (GitOps)

Duração
45 min

Pré-requisito
Labs 1 e 2

Objetivo do Laboratório

Separar o repositório de código do repositório de configuração (manifests), implementando o padrão **GitOps** onde o pipeline de CI atualiza o repositório de configuração e um pipeline separado aplica as mudanças no ambiente.

6. Boas Práticas de Design e Manutenção

Princípios Fundamentais

Fail fast

Lint e testes unitários primeiro;
E2E por último

Pipeline como código

YAML no repositório, revisado
em Pull Request (PR)

Builds determinísticos

Mesmo commit = mesmo
artefato gerado sempre

Secrets seguros

Nunca em texto plano; use o
gerenciador da plataforma

Artefatos imutáveis

SHA do commit como tag, nunca utilize
":latest"

Ambientes paritários

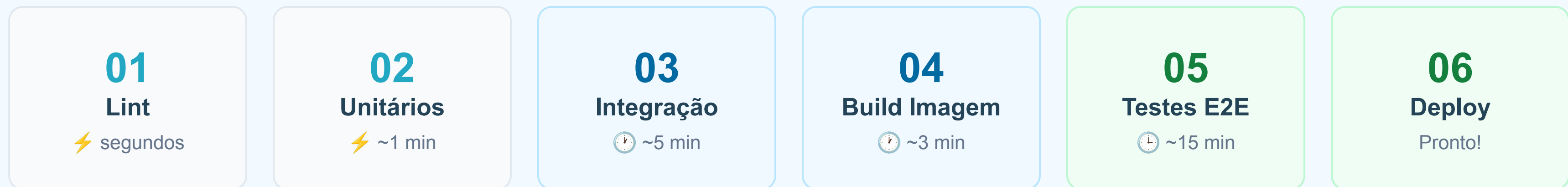
Dev/staging/prod o mais similares possível
entre si

Pipelines idempotentes

Rodar duas vezes = mesmo resultado final
garantido

Consistência e Velocidade: *A base para uma cultura de entrega contínua saudável.*

Ordem dos Estágios: Fail Fast



Princípio: *Se tem erro de sintaxe, descobre em segundos — não 40 minutos depois.*

Armadilhas Comuns

ARMADILHA

Pipeline que demora 40 min

SOLUÇÃO: Paralelizar jobs, cachear deps, separar PR/main

ARMADILHA

Testes flaky

SOLUÇÃO: Tratar como bug de alta prioridade, corrigir imediatamente

ARMADILHA

Ignorar builds quebrados

SOLUÇÃO: Build vermelho = prioridade #1; bloquear merges

ARMADILHA

YAML copiado entre repos

SOLUÇÃO: Reusable Workflows, Composite Actions, Shared Libs

ARMADILHA

Secrets com escopo maior

SOLUÇÃO: Princípio do menor privilégio

ARMADILHA

Nenhuma estratégia de rollback

SOLUÇÃO: Definir, documentar e testar o rollback previamente

Cultura CI/CD: Identificar gargalos e vulnerabilidades é o primeiro passo para a maturidade técnica.

Checklist do Pipeline Saudável

Design

- Pipeline versionado no repositório
- Estágios seguem a ordem fail-fast
- Jobs independentes em paralelo
- Tempo de pipeline de PR < 10 min

Qualidade e Segurança

- Nenhum secret hardcoded no YAML
- Imagens base sem `:latest`
- Dependências com versões fixadas

Deploy e Operação

- Health check automatizado pós-deploy
- Processo de rollback documentado
- Artefatos com SHA do commit

A automação robusta é a base para a entrega contínua com confiança.

Lab 04

CONTEXTO

Capítulo 6

Boas Práticas de Design e Manutenção de Pipelines

DURAÇÃO

40 min

PRÉ-REQUISITO

Labs 1 e 2 concluídos

OBJETIVO CENTRAL

Identificar problemas de design no pipeline e aplicar três melhorias concretas:

- **Cache de dependências:** Otimização de tempo.
- **Execução condicional:** Lógica de fluxo eficiente.
- **Templates reutilizáveis:** Manutenibilidade e escala.

Resumo da Unidade

Conceito Core

CI/CD

Resolve o "integration hell" com feedback rápido e releases frequentes.

Ecosystem

Plataformas

GitHub Actions, GitLab, Jenkins e Azure DevOps.

Fluxo

Pipeline

Build → Test → Deploy → Monitor com quality gates rigorosos.

Operação

GitOps

Git como fonte da verdade, modelo pull-based (ArgoCD/Flux).

Lançamento

Estratégias

Rolling, Blue-Green, Canary e Feature Flags.

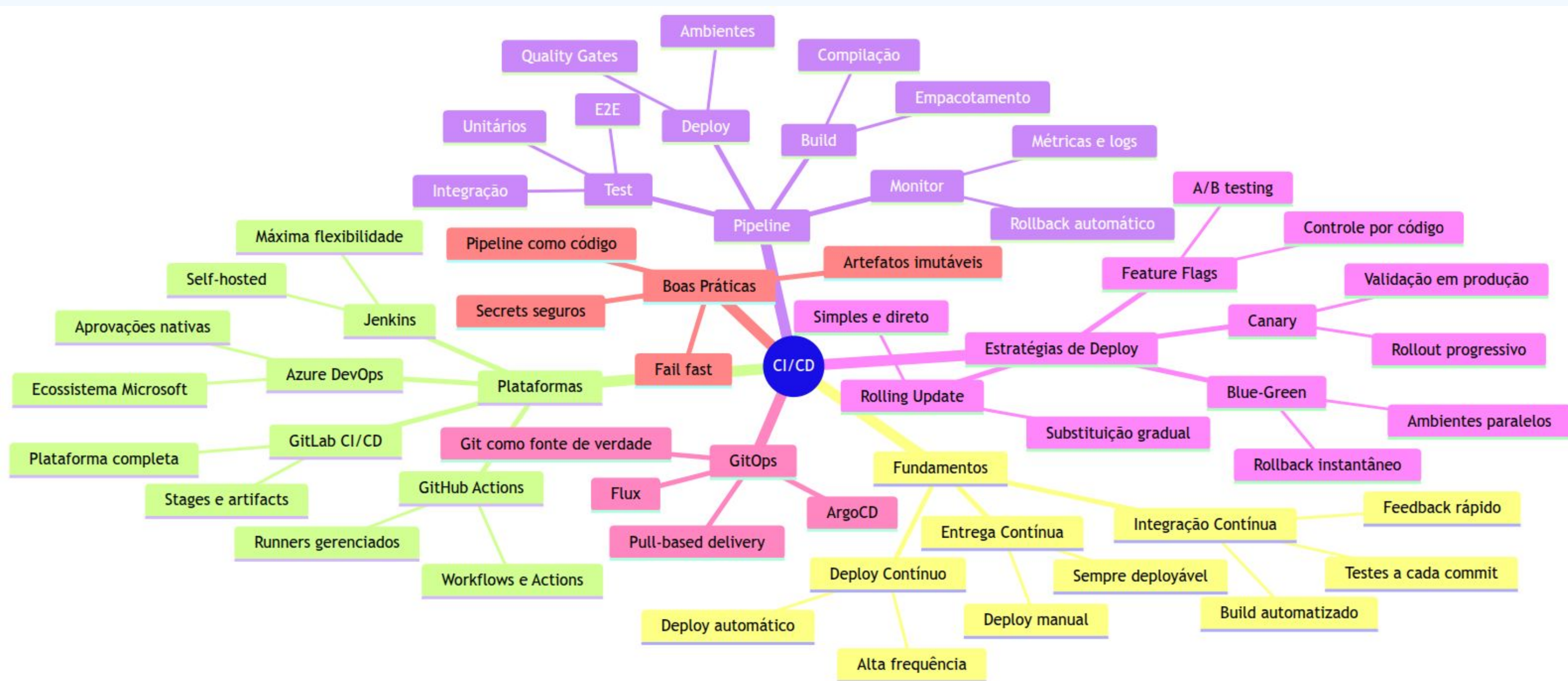
Fundamentos

Boas Práticas

Fail fast, secrets seguros, artefatos imutáveis e pipeline como código.

A maturidade em CI/CD é alcançada através da **automação, segurança e cultura de feedback.**

MAPA DE CONCEITOS



FRASES DE ENCERRAMENTO PARA REFLEXÃO DA UNIDADE

Integração Contínua

"Em time que está ganhando não se mexe. Em pipeline que está verde, muito menos."

Estratégias de Deployment

"Nunca faça deploy na sexta. E se fizer, que seja com fé, rollback configurado e o celular carregado."

Estágio de Deploy

"Quem tem um ambiente tem nenhum; quem tem três tem uma reunião de alinhamento."

OBRIGADO!!!